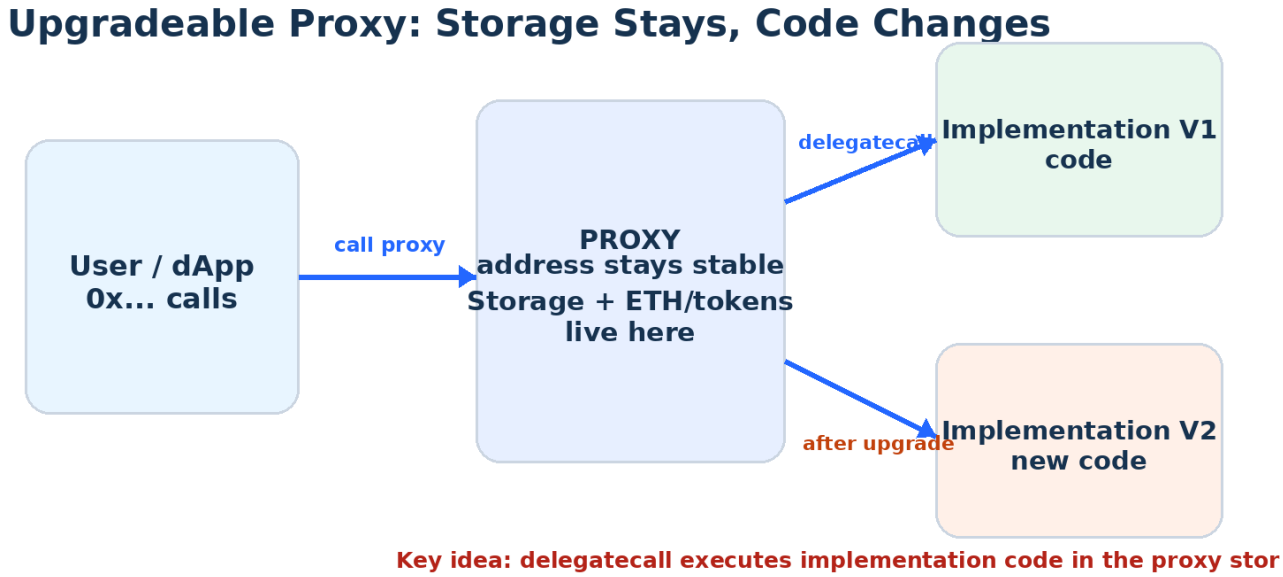
DAY 11

Smart Contract Security II: Upgradeable Architecture & Privileged Control

Proxies, UUPS, storage safety, admin design, timelocks, multisigs, oracle safeguards and secure upgrade operations

60-MINUTE LECTURE Beginner -> security-aware Solidity developer. Day 11 moves from isolated bugs to system-level risk: who can change code, how state survives upgrades, and how privileged operations are governed.



CAREER OUTCOME You should be able to inspect an upgradeable contract system, identify the proxy/implementation/admin structure, review storage compatibility, and explain a safer upgrade process to a client.

0-5 MIN

Learning objectives & 60-minute plan

What you should understand before touching an upgrade button.

By the end of Day 11, you should be able to:

- Explain proxy vs implementation and why delegatecall matters.
- Compare UUPS, Transparent and Beacon proxy patterns at a practical level.
- Use initializer-based setup and lock an implementation contract.
- Recognize unsafe storage reordering and safe append-only changes.
- Design upgrade authorization with roles, multisigs and timelocks.
- Validate a proposed upgrade before deployment and test state preservation.
- Apply freshness and sanity checks to oracle data.
- Write a concise client-facing security finding for upgrade/admin risk.

Minute	Topic
0-5	Day 10 recap + system threat model
5-12	Proxy / implementation / delegatecall
12-20	Proxy patterns + UUPS authorization
20-28	Initializers + storage layout
28-36	Roles, multisig, timelock, AccessManager
36-44	Secure upgrade workflow + validation
44-50	Oracle safety + privileged dependencies
50-56	Hands-on UUPS V1 -> V2 lab
56-60	Freelancer review, quiz, homework

DAY 10 RECAP Yesterday focused on direct bugs such as reentrancy and access mistakes. Today asks a higher-level question: even if the current code is safe, who can replace it tomorrow?

0-5 MIN

Threat model: immutable code vs upgrade authority

A non-upgradeable deployment has one major security advantage: after deployment, its runtime code does not change. An upgradeable system deliberately introduces a controlled path for changing behavior.

Design	Strength	New risk
Non-upgradeable	Simple trust model; code cannot be swapped through a proxy admin	Bug fixes may require migration or a new deployment
Upgradeable	Can patch bugs, add features and migrate behavior while preserving a stable proxy address	Upgrade authority becomes a critical trust boundary
Upgradeable + governance controls	Operational flexibility with review/approval delay	More moving parts; configuration mistakes can lock or expose control

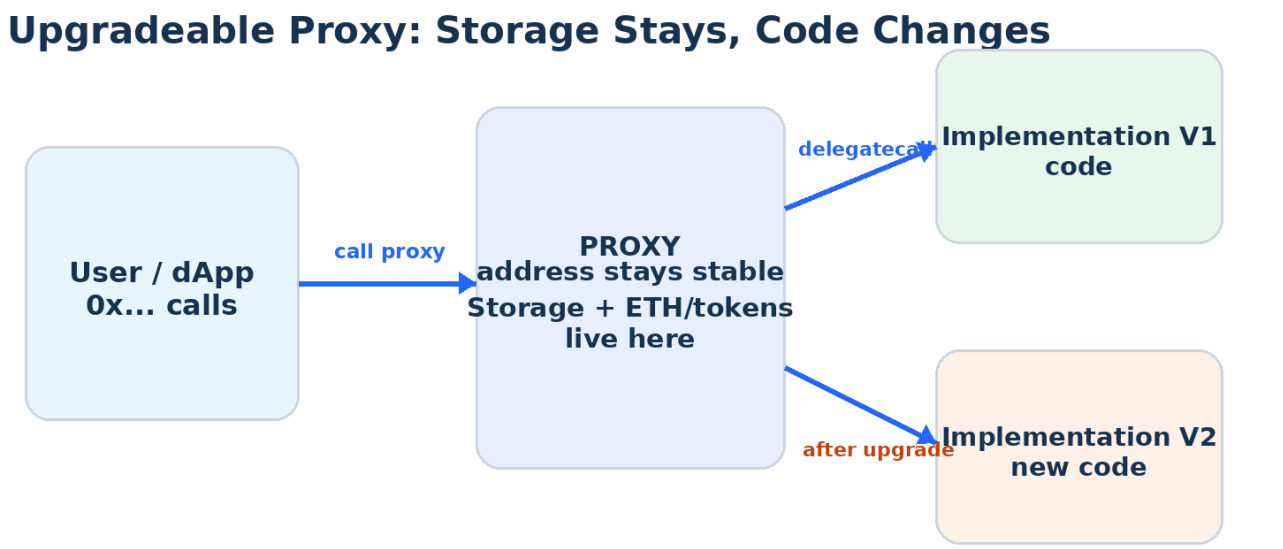
SECURITY QUESTION If an attacker gets the upgrade key, can they replace the implementation with code that transfers every token to themselves? In many systems, yes. Treat upgrade authority like control over the entire protocol.

System-level assets to map:

- Proxy address and implementation address
- Proxy type and upgrade mechanism
- Upgrade/admin role holders
- Timelock, multisig or governance contracts
- Initialization state and role configuration
- Storage layout across versions
- External dependencies: tokens, routers, oracles, bridges
- Emergency pause and recovery powers

5-12 MIN

Proxy mental model: stable storage, replaceable code



The user calls the proxy. The proxy delegates execution to an implementation. With delegatecall, the implementation code runs in the proxy context: storage writes affect the proxy, and the proxy address remains the address users interact with.

DO NOT CONFUSE Implementation storage is normally not the application state. The application state and assets are associated with the proxy. Calling the implementation directly is a different execution context.

```
// Conceptual proxy fallback (simplified)
fallback() external payable {
    address impl = _implementation();
    assembly {
        calldatacopy(0, 0, calldatasize())
        let ok := delegatecall(gas(), impl, 0, calldatasize(), 0, 0)
        returndatacopy(0, 0, returndatasize())
        switch ok
        case 0 { revert(0, returndatasize()) }
        default { return(0, returndatasize()) }
    }
}
```

INSTRUCTOR PROMPT Ask: If V1 stores totalDeposits in slot 0 and V2 interprets slot 0 as an address, what happens? This leads into storage layout safety.

12-20 MIN

Proxy patterns: UUPS, Transparent, Beacon and Clones

Pattern	Where upgrade logic lives	Typical use	Key review point
UUPS	Implementation exposes upgrade mechanism; proxy is ERC-1967 style	Single upgradeable app/protocol contracts	_authorizeUpgrade must be correctly restricted
Transparent	Proxy + ProxyAdmin separate admin calls from user calls	Common upgradeable deployments and tooling	ProxyAdmin ownership/permissions are critical
Beacon	Proxy reads implementation from a shared beacon	Many proxies that should upgrade together	One beacon upgrade can affect all attached proxies
Minimal proxy / clone	Usually fixed implementation reference; not automatically upgradeable	Cheap repeated instances	Do not assume clone == upgradeable proxy

Selection is an architecture decision, not a security badge.

- UUPS can reduce proxy complexity, but authorization is part of the implementation.
- Transparent proxies separate administrative dispatch from normal application calls.
- Beacon upgrades have a large blast radius because many proxies may share one implementation source.
- A client may use custom proxies; verify the actual bytecode and control path rather than guessing from naming.

FREELANCER HABIT On a block explorer, use proxy-read features and inspect implementation/admin slots or verified proxy metadata. Record exact addresses in your audit notes.

12-20 MIN

UUPS security: authorization belongs in the implementation

A UUPS implementation contains the upgrade function and must override the upgrade authorization hook. A missing or overly broad authorization policy is a critical design error.

```
bytes32 public constant UPGRADER_ROLE = keccak256("UPGRADER_ROLE");

function _authorizeUpgrade(address newImplementation)
    internal
    override
    onlyRole(UPGRADER_ROLE)
{}
```

Questions to ask during review:

- Who has UPGRADER_ROLE right now?
- Who can grant or revoke UPGRADER_ROLE?
- Is DEFAULT_ADMIN_ROLE held by a single EOA?
- Can the upgrade role be transferred accidentally?
- Is the upgrader a multisig, timelock, governor, AccessManager or EOA?
- Can the proposed implementation preserve UUPS compatibility?
- Are upgrade transactions monitored and documented?

CRITICAL The upgrade function being "only admin" is not enough analysis. You must inspect how that admin itself is controlled. Follow permissions recursively until you reach actual signers/governance.

20-28 MIN

Initializers: constructors do not initialize proxy storage

Upgradeable implementations normally use initializer functions because the implementation constructor runs only when the implementation itself is deployed. It does not initialize proxy storage.

```
contract FreelanceVaultV1 is
    Initializable,
    AccessControlUpgradeable,
    UUPSUpgradeable
{
    bytes32 public constant UPGRADER_ROLE = keccak256("UPGRADER_ROLE");

    constructor() {
        _disableInitializers(); // lock the implementation itself
    }

    function initialize(address admin) public initializer {
        __AccessControl_init();
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(UPGRADER_ROLE, admin);
    }

    function _authorizeUpgrade(address)
        internal override onlyRole(UPGRADER_ROLE) {}
}
```

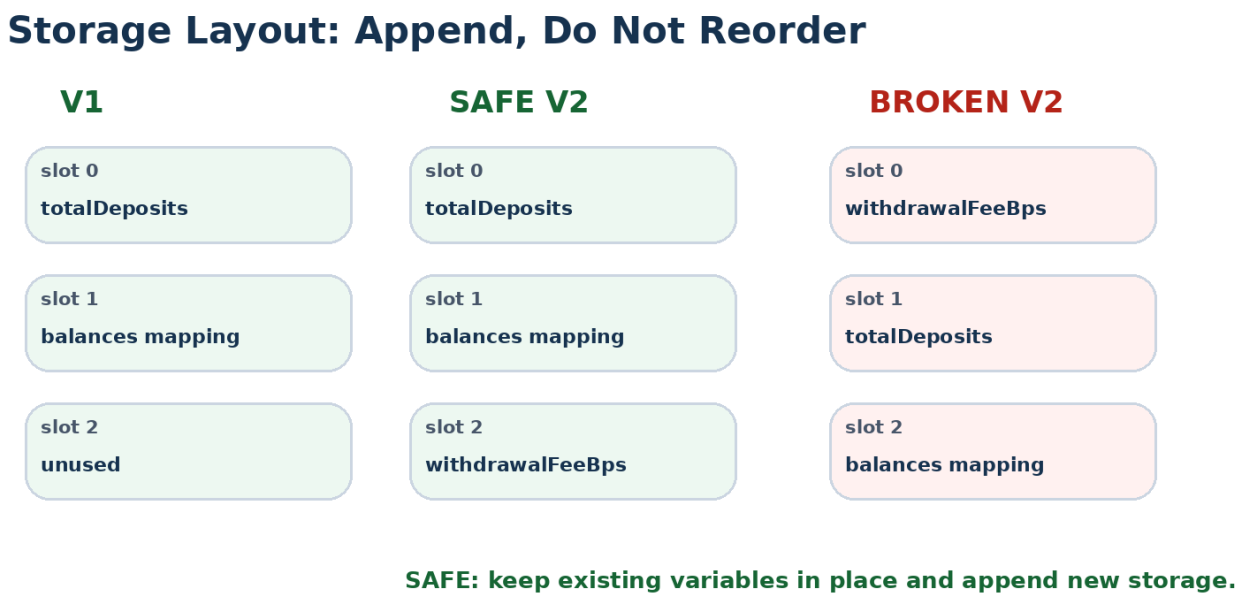
WHY LOCK IMPLEMENTATIONS? Disabling initializers on the implementation prevents someone from initializing that implementation address directly and claiming roles there. This is defense-in-depth for the logic contract.

Initializer review checklist:

- Proxy is initialized atomically during deployment where possible.
- Initializer cannot be called twice.
- All parent initializers required by inherited upgradeable modules are handled correctly.
- Admin/upgrader addresses are nonzero and expected.
- Implementation contract is locked against direct initialization.

20-28 MIN

Storage layout: the most dangerous invisible upgrade bug



Because the proxy keeps storage while the implementation changes, V2 must interpret old storage exactly the way V1 did. Reordering, deleting, or changing the type of existing state variables can reinterpret existing values.

SAFE DEFAULT Preserve existing storage order/types and append new state. Use upgrade validation tooling; do not rely on visual review alone, especially with inheritance.

```
// V1
uint256 public totalDeposits;               // existing storage
mapping(address => uint256) public balances; // existing storage

// V2 - safe simple change
uint256 public withdrawalFeeBps;            // appended new storage
address public feeRecipient;                 // appended new storage
```

20-28 MIN

Storage collision example: how funds accounting gets corrupted

Suppose V1 stores a number in slot 0. V2 incorrectly inserts a new address before it. The same 32 bytes are now decoded as a different variable. The proxy did not "lose" the bytes; the new code changed their meaning.

Version	slot 0	slot 1	Result
V1	totalDeposits = 100 ether	balances mapping seed	Correct
Bad V2	feeRecipient reads old numeric bytes	totalDeposits now reads mapping slot seed	Corrupted interpretation
Safe V2	totalDeposits unchanged	balances unchanged; new fields appended later	State preserved

MULTIPLE INHERITANCE Storage layout also depends on inheritance. A change to parent contracts can affect layout. This is why automated layout validation is essential for real upgrades.

Practical tests before approving V2:

- Record V1 balances, roles, totals and configuration.
- Upgrade on a local/forked environment.
- Read the same V1 state through V2.
- Run old behavior regression tests.
- Test new V2 behavior and migration/reinitializer logic.
- Validate storage layout with OpenZeppelin tooling.

28-36 MIN

Privileged control: Ownable, AccessControl and AccessManager

Security improves when privileges are explicit, minimal and separated. Avoid one key that can mint, pause, upgrade, withdraw and rewrite every parameter unless the system truly requires that trust model.

Tool / pattern	Use	Security question
Ownable / Ownable2Step	Simple single-owner administration	Should ownership be a multisig rather than a personal EOA?
AccessControl	Multiple roles with grant/revoke relationships	Who administers each role? Is DEFAULT_ADMIN_ROLE overpowered?
AccessManager	Centralized permission policy across multiple contracts, with role/function restrictions and delays	Does the authority map match the protocol architecture?
Custom auth	Application-specific permissions	Has the custom logic been independently reviewed and tested?

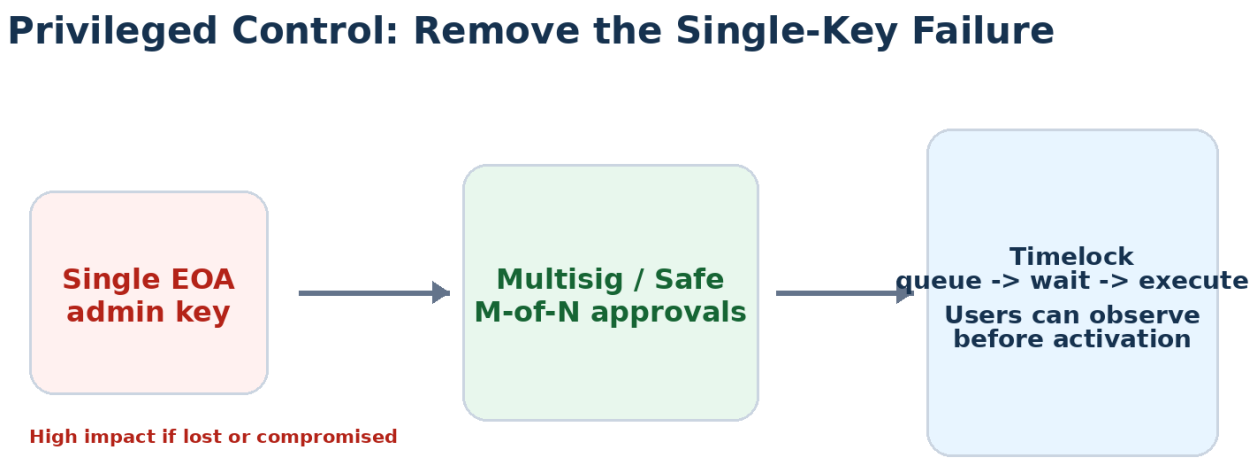
Example role separation:

```
UPGRADER_ROLE  -> multisig/timelock
PAUSER_ROLE     -> operations multisig
TREASURY_ROLE   -> finance multisig
PARAMETER_ROLE  -> limited risk manager
DEFAULT_ADMIN   -> highest-trust governance path
```

LEAST PRIVILEGE Give each actor only the functions they need. Separate emergency response from irreversible upgrades when possible.

28-36 MIN

Multisig + timelock: reduce key risk and give users reaction time



A multisig reduces dependence on one key by requiring multiple approvals. A timelock adds a delay between scheduling and execution. Together they can make high-impact changes more observable and harder to execute impulsively.

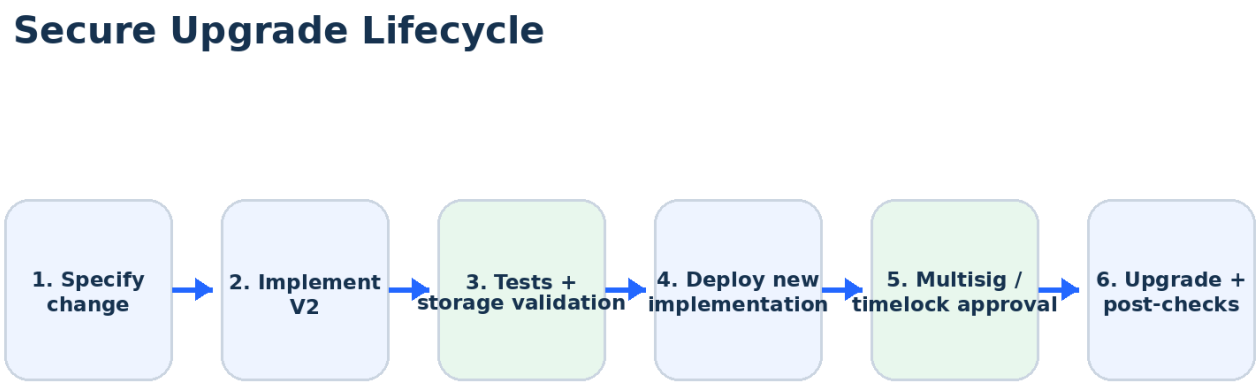
Typical high-impact operations to consider delaying:

- Implementation upgrades
- Changing oracle or router addresses
- Changing mint caps or treasury destinations
- Granting powerful roles
- Modifying liquidation/risk parameters
- Changing fee recipients or fee ceilings

TRADE-OFF Long delays improve reaction time but can slow emergency fixes. Production governance often separates routine/emergency actions and defines exactly which powers can bypass a delay.

36-44 MIN

Secure upgrade lifecycle: treat upgrades as releases



Never treat the upgrade transaction as the whole process: validation, approvals, monitoring, and

A professional upgrade is a controlled software release with blockchain-specific constraints. Do not jump from "V2 compiles" to "send upgrade transaction."

Release checklist:

- Write the intended behavior change and threat model.
- Review storage layout diff and inheritance changes.
- Run unit, fuzz, invariant and regression tests.
- Run an upgrade test from real V1 state or a mainnet fork when appropriate.
- Validate the implementation with upgrade tooling.
- Deploy implementation and verify source code.
- Obtain multisig/timelock/governance approvals.
- Execute upgrade, then run post-upgrade reads and canary transactions.
- Monitor events, balances, oracle health and privileged role changes.
- Document rollback/migration strategy before a problem occurs.

36-44 MIN

Upgrade validation with OpenZeppelin tooling

The upgrades tooling can validate implementation safety and compare storage layouts before an actual upgrade. Treat "unsafe" bypass flags as exceptional, review-heavy decisions rather than convenience switches.

```
// Hardhat 3 style - simplified example
import hre from "hardhat";
import { upgrades } from "@openzeppelin/hardhat-upgrades";

const connection = await hre.network.create();
const { ethers } = connection;
const upgradesApi = await upgrades(hre, connection);

const V2 = await ethers.getContractFactory("FreelanceVaultV2");
await upgradesApi.validateUpgrade(PROXY_ADDRESS, V2, { kind: "uups" });

// Only after validation, testing and approvals:
await upgradesApi.upgradeProxy(PROXY_ADDRESS, V2, { kind: "uups" });
```

RED FLAG unsafeSkipStorageCheck disables a major safety check. A client asking you to "just bypass the error" is a reason to investigate the incompatibility, not a reason to proceed.

What validation cannot replace:

- Business-logic review
- Authorization/governance review
- Oracle and external integration review
- Economic attack analysis
- Operational monitoring
- Independent audit for high-value systems

50-56 MIN

Hands-on lab: FreelanceVault V1 (UUPS)

Goal: deploy V1 behind a UUPS proxy, deposit test ETH, record state, then upgrade to V2 without changing the proxy address or corrupting balances.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import {Initializable} from
  "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import {AccessControlUpgradeable} from
  "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";
import {UUPSUpgradeable} from
  "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";

contract FreelanceVaultV1 is Initializable, AccessControlUpgradeable, UUPSUpgradeable {
    bytes32 public constant UPGRADER_ROLE = keccak256("UPGRADER_ROLE");
    uint256 public totalDeposits;
    mapping(address => uint256) public balances;

    constructor() { _disableInitializers(); }

    function initialize(address admin) public initializer {
        __AccessControl_init();
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(UPGRADER_ROLE, admin);
    }

    function deposit() external payable {
        balances[msg.sender] += msg.value;
        totalDeposits += msg.value;
    }

    function withdraw(uint256 amount) external {
        require(balances[msg.sender] >= amount, "insufficient");
        balances[msg.sender] -= amount;
        totalDeposits -= amount;
        (bool ok,) = payable(msg.sender).call{value: amount}("");
        require(ok, "transfer failed");
    }

    function _authorizeUpgrade(address) internal override onlyRole(UPGRADER_ROLE) {}
}
```

50-56 MIN

Hands-on lab: V2 adds fee configuration safely

V2 inherits V1 and appends new storage. Existing V1 state must remain readable after the upgrade.

```
contract FreelanceVaultV2 is FreelanceVaultV1 {
    uint256 public withdrawalFeeBps; // appended storage
    address public feeRecipient;     // appended storage

    function initializeV2(address recipient, uint256 feeBps)
        external reinitializer(2) onlyRole(DEFAULT_ADMIN_ROLE)
    {
        require(recipient != address(0), "zero recipient");
        require(feeBps <= 500, "fee too high"); // <= 5% example policy
        feeRecipient = recipient;
        withdrawalFeeBps = feeBps;
    }

    function version() external pure returns (uint256) {
        return 2;
    }
}
```

LAB NOTE The sample focuses on upgrade mechanics, not fee-charging production logic. A real fee-bearing withdraw path needs careful accounting, CEI/reentrancy review, tests, and explicit economic requirements.

State preservation assertions:

```
assertEq(address(proxy), sameProxyAddress);
assertEq(v2.totalDeposits(), totalBefore);
assertEq(v2.balances(alice), aliceBefore);
assertTrue(v2.hasRole(v2.UPGRADER_ROLE(), admin));
assertEq(v2.version(), 2);
```

50-56 MIN

Lab procedure & negative tests

1. Deploy V1 implementation and a UUPS proxy; initialize the proxy admin role.
2. Send test deposits from two test accounts.
3. Record proxy address, implementation address, balances, totalDeposits and role holders.
4. Compile V2 and run storage/upgrade validation.
5. Attempt an upgrade from an unauthorized account - it must revert.
6. Upgrade from the authorized governance path.
7. Call initializeV2 once; a second call must revert.
8. Verify all V1 state is unchanged.
9. Exercise V1 deposit/withdraw behavior again through V2.
10. Verify new V2 getters/logic and emitted upgrade events.

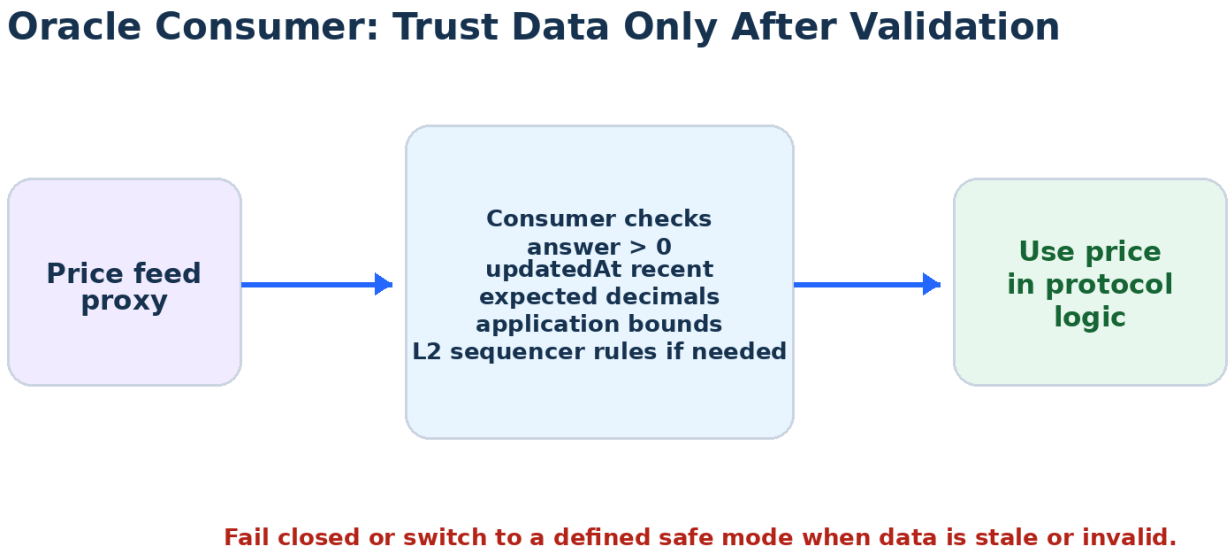
Negative tests that matter:

- Unauthorized caller cannot upgrade.
- Implementation address cannot be initialized directly.
- Bad V2 with reordered storage fails validation.
- Initializer/reinitializer cannot run twice.
- New implementation preserves interface and old user flows.
- Upgrade to an incompatible/non-UUPS implementation is rejected by tooling/on-chain checks as applicable.

PORTFOLIO IDEA Publish the lab with V1/V2 contracts, test output, a storage-layout screenshot/diff, and a short upgrade runbook. This is much stronger than a repository containing only deployable token code.

44-50 MIN

Oracle security: external data is another privileged dependency



An oracle can be cryptographically and operationally robust but still be used unsafely by your contract. The consumer must define acceptable freshness and validity for its own application.

Consumer-side checks:

- Read from the intended feed proxy/address for the correct network and asset pair.
- Reject non-positive or otherwise invalid answers where the domain requires it.
- Check updatedAt / latest timestamp against an application-specific maximum age.
- Understand feed decimals before converting values.
- Define reasonable application limits and behavior during extreme volatility.
- On L2 applications, consider sequencer uptime/grace-period requirements when relevant.
- Monitor feed health and define a pause/safe-mode plan.

DO NOT HARDCODE RANDOM FEEDS Verify feed addresses from the official network-specific catalog. A correct ABI connected to the wrong feed can produce perfectly valid but economically wrong data.

44-50 MIN

Oracle consumer code: freshness + sanity checks

```
interface AggregatorV3Interface {
    function decimals() external view returns (uint8);
    function latestRoundData() external view returns (
        uint80 roundId,
        int256 answer,
        uint256 startedAt,
        uint256 updatedAt,
        uint80 answeredInRound
    );
}

contract SafePriceConsumer {
    AggregatorV3Interface public immutable feed;
    uint256 public immutable maxAge;

    error InvalidPrice();
    error StalePrice();

    constructor(address feed_, uint256 maxAge_) {
        feed = AggregatorV3Interface(feed_);
        maxAge = maxAge_;
    }

    function price() external view returns (uint256 value, uint8 decimals_) {
        (, int256 answer,, uint256 updatedAt,) = feed.latestRoundData();
        if (answer <= 0) revert InvalidPrice();
        if (updatedAt == 0 || block.timestamp - updatedAt > maxAge) revert StalePrice();
        return (uint256(answer), feed.decimals());
    }
}
```

IMPORTANT maxAge is an application risk parameter. It should reflect the feed characteristics and the economic consequences of stale data; it is not a universal constant.

SECURITY SCENARIOS

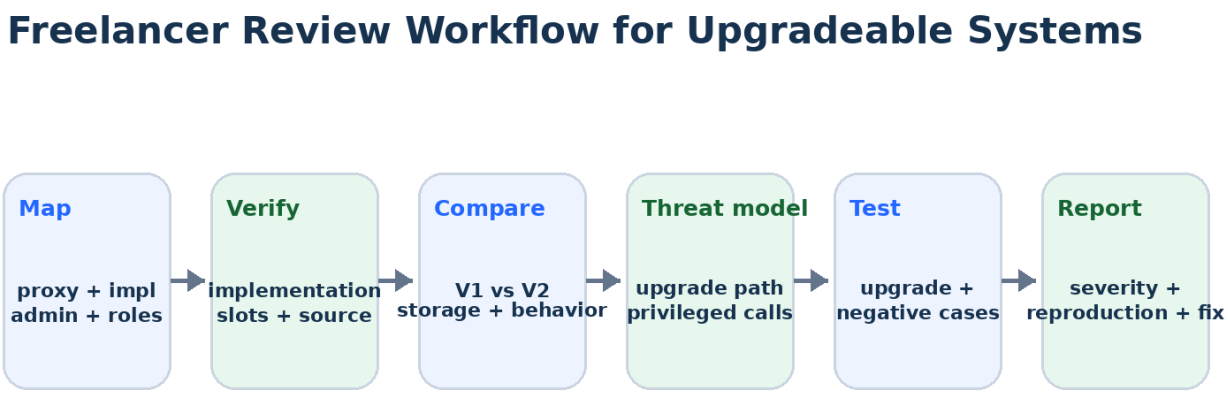
Failure modes that look like "admin features" until they fail

Scenario	Impact	Review / mitigation
Single hot-wallet upgrader compromised	Attacker can deploy malicious implementation	Move upgrade authority to appropriate multisig/timelock; monitor changes
V2 reorders storage	Balances/roles/config may be corrupted	Storage validation + upgrade regression tests
Proxy deployed uninitialized	Attacker may initialize and seize roles	Initialize atomically; verify initialization state
Implementation left initializable	Logic address can be initialized directly	_disableInitializers in implementation constructor
Timelock has no viable proposer/executor	System can become operationally locked	Role/recovery design + documented governance runbook
Oracle data accepted without freshness check	Protocol may operate on stale price	updatedAt/max-age checks + monitoring/safe mode
Emergency role can also upgrade	Response key has excessive blast radius	Separate operational and upgrade privileges

SEVERITY THINKING Severity depends on impact + exploitability + current configuration. A dangerous function that no attacker can call today may still be a governance/design risk if an overly powerful role can grant access instantly.

56-60 MIN

Client audit workflow: produce evidence



Deliver evidence, not guesses: addresses, roles, storage diffs, tests, transaction traces, and cc

A professional review traces the entire upgrade trust chain rather than reading only the implementation source.

Finding template:

Field	Example
Title	Upgrade authority held by a single externally owned account
Severity	High - depends on TVL, controls and operational context
Affected components	Proxy 0x..., implementation 0x..., UPGRADER_ROLE holder 0x...
Evidence	Role query, proxy implementation slot, transaction history, verified source
Impact	Compromise of one key may permit arbitrary implementation replacement
Recommendation	Use appropriate multisig/timelock/governance; separate roles; add monitoring and runbook
Validation	Test unauthorized upgrades; verify new governance addresses and permissions

FREELANCER PRACTICE

Upwork / LinkedIn client scenarios

Scenario A - "Can you tell whether this contract is upgradeable?"

- Inspect bytecode/source and explorer proxy metadata.
- Identify proxy type, implementation and admin/control address.
- Read EIP-1967-related slots/tooling where applicable.
- Explain whether users interact with proxy or implementation.

Scenario B - "Our upgrade validator reports incompatible storage."

- Do not bypass the check first.
- Diff V1/V2 state variables and inherited contracts.
- Find reorder/type/delete/parent-layout changes.
- Refactor V2 to preserve layout, then rerun tests and validation.

Scenario C - "We want the owner wallet to upgrade instantly."

- Clarify TVL, operational needs, incident response and governance expectations.
- Explain the single-key risk and alternatives such as multisig/timelock.
- Document the chosen trust model explicitly if the client accepts the risk.

CLIENT COMMUNICATION Security consulting is not just saying "this is bad." State the exact control path, plausible impact, trade-offs, and a practical remediation plan.

KNOWLEDGE CHECK

Interview questions + 12-question quiz

Interview questions

1. Why does a proxy use delegatecall?
2. Where is application storage in a typical proxy system?
3. Why are constructors replaced by initializers?
4. What does _disableInitializers protect?
5. Why is storage layout compatibility critical?
6. What is the core UUPS authorization hook?
7. Transparent vs UUPS: what is a major architectural difference?
8. Why can Beacon upgrades have a large blast radius?
9. How can a multisig reduce admin risk?
10. What does a timelock add beyond multisig?
11. Why should oracle updatedAt be checked?
12. What should you verify after an upgrade?

Mini quiz - choose the best answer:

1. Safe V2 storage change? A reorder fields B append new field C change uint to address
2. UUPS upgrade authorization? A tokenURI B _authorizeUpgrade C receive
3. Proxy state lives primarily at? A proxy B compiler C ABI file
4. Best response to storage validation failure? A skip it B investigate incompatibility C force upgrade
5. Oracle freshness field? A updatedAt B symbol C owner name
6. Timelock primarily adds? A faster execution B delay/observability C gas refund

REVIEW

Quiz answers, homework & portfolio task

Quiz answers

- 1: B - append new storage
- 2: B - _authorizeUpgrade
- 3: A - proxy
- 4: B - investigate incompatibility
- 5: A - updatedAt
- 6: B - delay/observability

Homework (45-90 minutes)

11. Create or clone the V1/V2 lab in a local project using test accounts only.
12. Write an upgrade test that deposits before the upgrade and checks all balances after V2 is active.
13. Create a deliberately broken V2 by reordering a variable; capture the validation error, then fix it.
14. Add an unauthorized-upgrader negative test.
15. Write a one-page security finding for a hypothetical single-EOA upgrader controlling a high-value vault.
16. Build a small oracle consumer test: fresh price passes; stale timestamp must revert.

Portfolio deliverable

- Architecture diagram showing proxy, implementation and admin path
- V1 + V2 source
- Automated upgrade/storage tests
- Before/after state snapshot
- Upgrade runbook
- One sample security finding
- README explaining trust assumptions

CHEAT SHEET

Day 11 one-page reference + primary sources

Concept	Remember
Proxy	Stable user-facing address; delegates execution
Implementation	Code executed through proxy context
delegatecall	Implementation code, proxy storage/context
Initializer	Sets proxy state instead of relying on implementation constructor
Storage safety	Preserve old layout; append new storage; validate automatically
UUPS	Implementation carries upgrade mechanism; restrict _authorizeUpgrade
AccessControl	Separate roles; inspect role admins
Multisig	Multiple approvals reduce single-key risk
Timelock	Delay high-risk operations so they are observable
Oracle	Validate address, answer, decimals and freshness
Upgrade test	Preserve old state + old behavior + new behavior
Post-upgrade	Verify implementation, roles, state, events and monitoring

Primary technical references

OpenZeppelin Contracts 5.x - Proxy API: https://docs.openzeppelin.com/contracts/5.x/api/proxy

OpenZeppelin Upgrades - Writing Upgradeable Contracts: https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable

OpenZeppelin Contracts 5.x - Access Control: https://docs.openzeppelin.com/contracts/5.x/access-control

OpenZeppelin Hardhat Upgrades API: https://docs.openzeppelin.com/upgrades-plugins/api-hardhat-upgrades

Chainlink Data Feeds: https://docs.chain.link/data-feeds

EIP-1967 Proxy Storage Slots: https://eips.ethereum.org/EIPS/eip-1967

EIP-1822 Universal Upgradeable Proxy Standard: https://eips.ethereum.org/EIPS/eip-1822

DAY 12 PREVIEW Web3 Frontend Development: providers, signers, ABI/contract instances, wallet connection, read calls, write transactions, receipts, errors and a complete ethers.js dApp flow.